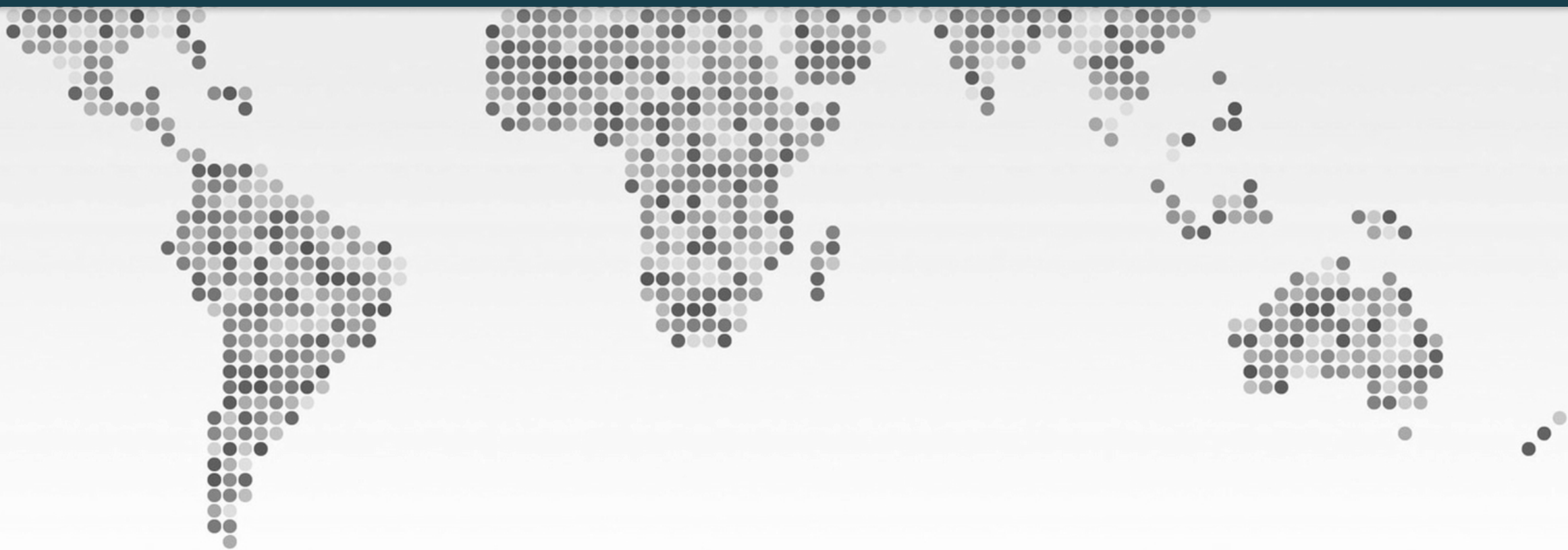


Software Engineering for Artificial Intelligence

Quality Assurance of Artificial Intelligence Solutions



Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Basics

Quality assurance measures the level of quality in a service/product by means of manual/automated mechanisms at every stage of the process of delivery or production.

QA approaches for software can be roughly divided into three large groups:

- 1 *Dynamic approaches* execute the software, including traditional testing and dynamic program analyses.
- 2 *Static approaches* analyze code, documentation, designs, or other software artifacts without executing them, including code review, code linting, static data-flow analysis, and formal verification.
- 3 *Process evaluation approaches* analyze how the software was produced rather than the software artifacts, for example, assuring that requirements are documented and known bugs are tracked.

Many QA activities can be automated. For example, the execution large test suites automatically after every change to a code base. At the same time, some QA approaches rely on manual steps or are entirely manual, such as asking experts to review design documents for performance bottlenecks or security weaknesses.

QA often focuses on *functional correctness* to evaluate whether the software produces the expected outputs according to some specification. Beyond functional correctness, there are many other *qualities of interest* for components or the system as a whole.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Basics

In the context of this lecture, we will mainly focus on *static approaches*, to understand the typical issues arising in AI-enabled systems that can be only diagnosed by means of manual or automated static analysis of source code.

At first, *code review*, traditionally also known as inspection: this is the process of manually reading and analyzing code or other artifacts to look for problems.

Today, the most common form of code review is *lightweight incremental code review* of changes when *they are committed into a version control system*. Essentially all big tech companies have adopted some code review process and tooling where each set of changes needs to be reviewed and approved by multiple others before it will be integrated into the shared code base.

In open source, this is commonly achieved through reviewing *pull requests* before they are merged into the main development branch.

Contemporary code review is only moderately effective at finding defects, but *it can surface other issues that are difficult to test for*, such as poor code readability, poor documentation, bad coding patterns, poor extensibility, and inefficient code.

In an AI context, code reviews can be extended to all code and artifacts of the AI pipeline, including data-wrangling code, model training code, model evaluation reports, model requirements, model documentation, deployment infrastructure, configuration code, and monitoring plans.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Basics

Static analysis describes a family of approaches to automatically reason about code without executing it. There are many different static analyses, typically categorized and named after the technical flavor of the underlying technique, such as *type checking*, *code linting*, *data-flow analysis*, or *information-flow analysis*.

Static analyses commonly report style issues (e.g., violating naming conventions), suspicious but not technically illegal code (e.g., assignment within an if expression), and clear mistakes (e.g., dead code, calls to undefined functions).

Most static analyses rely on heuristics that point out common problems but may miss other problems and may raise false alarms - the high false positive rate is normally considered the most problematic issue of contemporary static analysis tools.

For dynamically typed and highly flexible languages such as *Python* and *JavaScript*, static analysis tools are usually weaker than those for *Java* or *C*; as such, they tend to focus on style issues and simple problem patterns.

In any case, using static analysis is a common pattern in industry: essentially, they **can enforce coding standards**, other than improve maintainability, and reduce technical debt by providing immediate feedback to developers as they write code.

To be effective, these activities need to be planned, automated where possible, and integrated into the development process. Typically, static analysis tools are introduced and configured within CI pipelines, so that they can check code continuously.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

1

Quantity and quality of data. More data leads to better models. However, this is only true up to a point—there are diminishing effects of adding more data.

In terms of quality, it is useful to distinguish imprecise data (random noise) from inaccurate data (systematic problem). Imprecise data from a noisy measurement process may lead to less confident models and occasionally spurious predictions if the model overfits to random measurement noise. However, noise in training data is something that most AI techniques can handle quite well.

In contrast, inaccurate data is much more dangerous, because it will lead to misleading models. For example, a model trained on sales data with a miscalibrated scale systematically underrepresenting the sales of bananas will predict a too-low need for ordering bananas.

Inaccurate data with systematic biases rather than noisy data is the source of most fairness issues, as we will further discuss in the remainder of the course.

In any case, both data quantity and data quality are important. Given a limited budget, some trade-off analysis must be put into place.

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

2

The myth of raw data. When data represents real-world phenomena, it is always an abstraction. Somebody has made a decision about what data to collect, how to encode the data, and how to organize it for storage and processing.

For example, when we record how many bananas are in our inventory, we need to decide whether we record weight, number, or size, whether and how we record ripeness, and whether to record the country of origin, the supplier, or a specific plantation. Even when data is used directly from a sensor, the designer of the sensor has made decisions about how to observe real-world phenomena and encode them.

As such, data collection is always necessarily selective and interpretative. It is never raw and objective. Arguably, “*raw data*” is an oxymoron. At most, we can consider data as raw with regard to somebody else’s collection decisions.

On the one hand, this implies that the accuracy of data can only be evaluated with regard to a specific operationalization of a measurement process. On the other hand, the overall soundness of the AI process may be intrinsically biased.

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

- 3 *Data quality criteria.* Because of the importance of data quality, multiple criteria exist to evaluate the intrinsic properties of data - some should be already known.

Criterion	Definition	Rationale
Accuracy	The data was recorded correctly.	Inaccurate data can lead to incorrect analyses, decisions, or predictions.
Completeness	All relevant data was recorded.	Incomplete data can result in biased or misleading outcomes.
Uniqueness	All entries are recorded once.	Duplicate records can skew results and waste resources.
Consistency	The data agrees with each other.	Inconsistent data across systems or records can cause confusion and errors.
Currentness	The data is kept up to date.	Outdated data can lead to irrelevant or incorrect decisions.

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

- 3 *Data quality criteria.* Because of the importance of data quality, multiple criteria exist to evaluate the intrinsic properties of data - some should be already known. These criteria can be associated with actionable metrics, e.g., the number of non-empty values over the total amount of values may measure completeness. Perhaps more importantly, these criteria are associated with the so-called **data smells**.

Data smell. A subtle indicator of potential issues in a dataset, such as inconsistencies, anomalies, or patterns that suggest poor data quality or design.

Data smells violate data quality criteria and may lead to subsequent training issues.

For instance, let us consider the case of the *Extreme Value* smell. This refers to the presence of unusually large or small values in a dataset that deviate significantly from the norm, often indicating outliers or errors.

It impacts accuracy, as it may result from incorrect data entry or faulty sensors. The smell may distort the learning process and cause instability.

Software Engineering for Artificial Intelligence: Quality Assurance

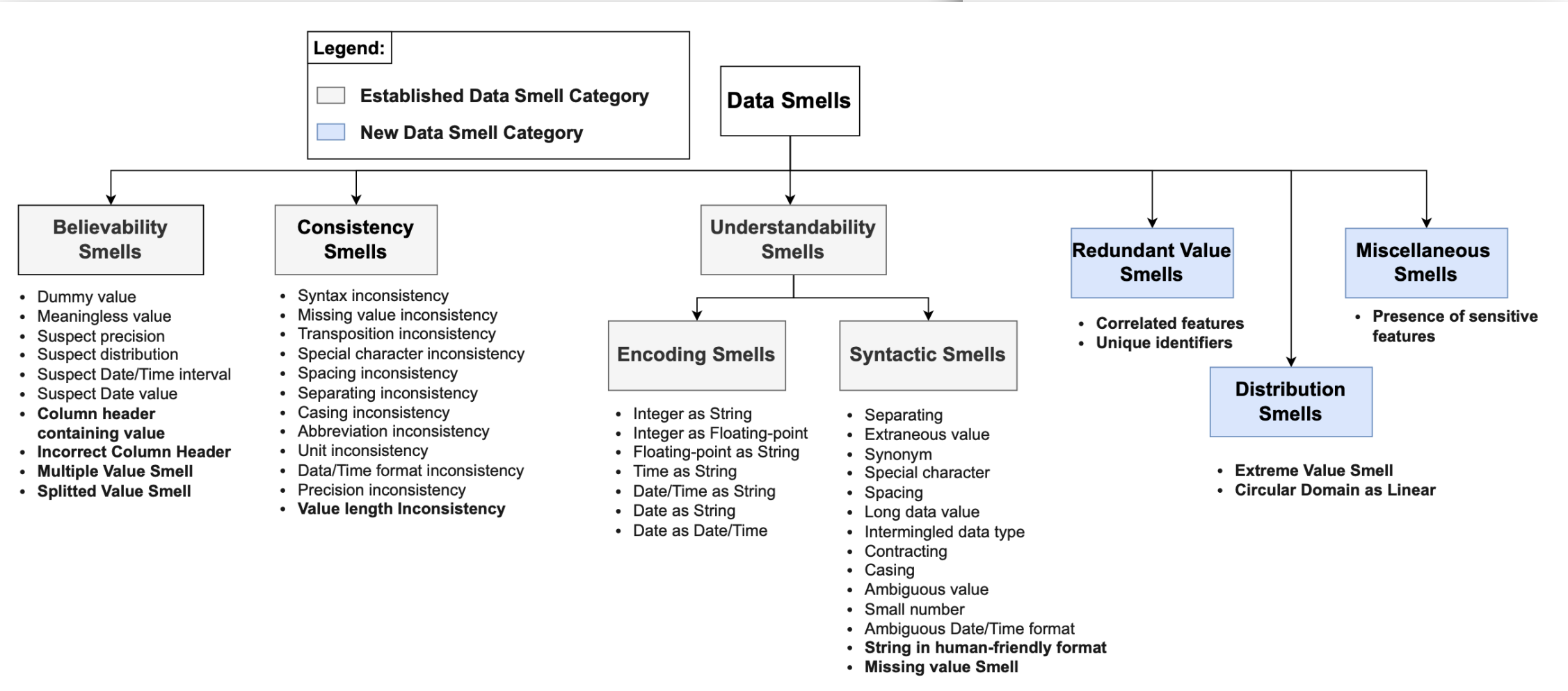
Quality Assurance of Artificial Intelligence Solutions: What to Look For

Data smell. A subtle indicator of potential issues in a dataset, such as inconsistencies, anomalies, or patterns that suggest poor data quality or design.

Unmasking Data Secrets: An Empirical Investigation into Data Smells and Their Impact on Data Quality

Gilberto Recupito
Sesa Lab - University of Salerno
Salerno, Italy
grecupito@unisa.it
Dario Di Nucci

Raimondo Rapacciuolo
University of Salerno
Salerno, Italy
r.rapacciuolo1@studenti.unisa.it
Fabio Palomba



used nowadays, be-
informed decisions
one by humans [37].
embed artificial in-
recently deployed in
ons showing highly

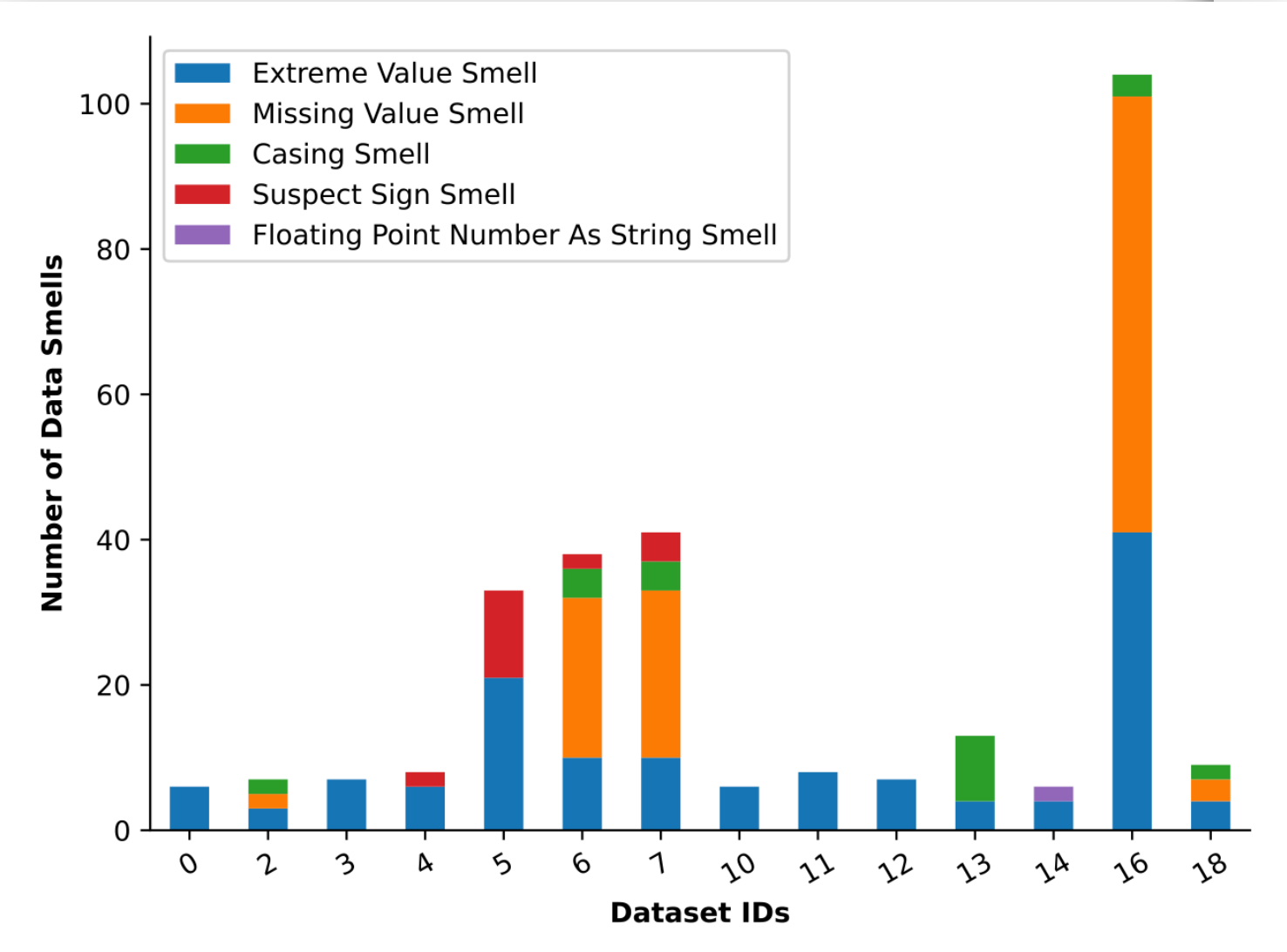
igence-enabled sys-
se the program and
ily rely on the data

defined as a system
of which at least one
ext, data represents
ited AI-enabled sys-
n is a crucial initial
nts. Failure to prop-
el degradation [26].
y to build a reliable
issues can arise for
uate data cleaning,
quires a well-defined
profiling, cleansing,
ls and practices are
ata transformation
prove the practices
creasing [9].
chnical debt for the
introduced by data
impact AI-enabled
ausing problems to
line [1].
ata smells are repre-
e smells are defined
n choices [14], data
t data quality issues
ns in the future [13].
stigated in research
about the effect and
e catalog provided

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Data smell. A subtle indicator of potential issues in a dataset, such as inconsistencies, anomalies, or patterns that suggest poor data quality or design.



Unmasking Data Secrets: An Empirical Investigation into Data Smells and Their Impact on Data Quality

Gilberto Recupito
Sesa Lab - University of Salerno
Salerno, Italy
grecupito@unisa.it

Dario Di Nucci
Sesa Lab - University of Salerno
Salerno, Italy
ddinucci@unisa.it

Raimondo Rapacciuolo
University of Salerno
Salerno, Italy
r.rapacciuolo1@studenti.unisa.it

Fabio Palomba
Sesa Lab - University of Salerno
Salerno, Italy
fpalomba@unisa.it

ABSTRACT

Artificial Intelligence (AI) is rapidly advancing with a data-centered approach suitable for various domains. Nevertheless, AI faces significant challenges, particularly in data quality. Data collection from diverse sources can introduce quality issues that may threaten the performance of AI-enabled systems. A growing concern in this context is the emergence of *data smells* – issues specific to the data used in building AI models, which can have long-term consequences. In this paper, we aim at enlarging the current body of knowledge on data smells, by proposing a two-step investigation on the matter. First, we updated an existing literature review in order to cataloguing the currently existing data smells and the methods to detect them. Afterward, we assess the prevalence of data smells and their correlation with data quality metrics. We identify a dataset composed of 12 data smells distributed across three main categories. Secondly, we observe that the correlation between data smells and data quality is notably impactful, exhibiting a pronounced and substantial effect, especially in highly diffused smell instances. This research sheds light on the complex relationship between data smells and data quality, providing valuable insights into the challenges of maintaining AI-enabled systems.

CONCEPTS

Software engineering → Software maintenance tools.

KEYWORDS

Technical Debt; Data Smells; Data Quality; Software Engineering for Artificial Intelligence, Empirical Software Engineering.

Reference Format:

Gilberto Recupito, Raimondo Rapacciuolo, Dario Di Nucci, and Fabio Palomba. Unmasking Data Secrets: An Empirical Investigation into Data Smells and Their Impact on Data Quality. In *Conference on AI Engineering Software Engineering for AI (CAIN 2024)*, April 14–15, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3644815.3644960>

CC BY licensed under Creative Commons Attribution International 4.0 License.

24, April 14–15, 2024, Lisbon, Portugal
Copyright held by the owner/author(s).
ISBN 979-8-4007-0591-5/24/04.
<https://doi.org/10.1145/3644815.3644960>

1 INTRODUCTION

Artificial Intelligence (AI) is more and more diffused nowadays, being used by individuals and companies to make informed decisions [54] and automate tasks that would typically be done by humans [37]. Indeed, AI-intensive systems, i.e., systems that embed artificial intelligence models and algorithms, have been recently deployed in multiple domains, with some recent applications showing highly efficient and accurate performance [30, 33].

However, the development of artificial intelligence-enabled systems differs from other types of software because the program and its effectiveness in solving a specific task heavily rely on the data and observations used to train models [2].

More specifically, AI-enabled systems are defined as a system consisting of various software components, out of which at least one is an AI-specific component [29]. In this context, data represents the primary source of producing business-oriented AI-enabled systems. Performing data analysis and validation is a crucial initial step for designers of machine learning components. Failure to properly analyze the training data may lead to model degradation [26]. Therefore, it is crucial to prioritize data quality to build a reliable and effective AI-enabled system. Data quality issues can arise for various reasons, e.g., data entry errors, inadequate data cleaning, or bias in the data. Addressing these issues requires a well-defined data quality management strategy that includes profiling, cleansing, and data enrichment [32]. While different tools and practices are available to support feature engineering and data transformation for managing AI pipelines [38], the need to improve the practices related to quality assurance is continuously increasing [9].

Data quality degradation could also lead to technical debt for the whole system [12]. Data debt, primarily when introduced by data quality issues or data anomalies, can strongly impact AI-enabled systems, degrading model performance and causing problems to all the subsequent phases involved in the pipeline [1].

In analyzing technical debt specific to data, *data smells* are represented using the analogy of code smells. As code smells are defined as symptoms of poor design and implementation choices [14], data smells are data value-based indications of latent data quality issues caused by poor practices that may lead to problems in the future [13]. While other types of data quality issues are investigated in research to be addressed [16, 18, 27, 49], the knowledge about the effect and the presence of data smells is still limited to the catalog provided by Foidl et al. [13].

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Data smell. A subtle indicator of potential issues in a dataset, such as inconsistencies, anomalies, or patterns that suggest poor data quality or design.

While individual data smells may exhibit a seemingly low impact, the significance levels associated with each correlation signal the severity of having these smells in high quantities.

In other terms, data smells may actually affect multiple data quality criteria, potentially lowering the overall usefulness and suitability of the data.

As such, we may want to have static analysis instruments that help data engineers/scientists track the presence of data smells; also, code review activities may focus on finding these defects.

Unmasking Data Secrets: An Empirical Investigation into Data Smells and Their Impact on Data Quality

Gilberto Recupito
Sesa Lab - University of Salerno
Salerno, Italy
grecupito@unisa.it

Dario Di Nucci
Sesa Lab - University of Salerno
Salerno, Italy
ddinucci@unisa.it

Raimondo Rapacciuolo
University of Salerno
Salerno, Italy
r.rapacciuolo1@studenti.unisa.it

Fabio Palomba
Sesa Lab - University of Salerno
Salerno, Italy
fpalomba@unisa.it

ABSTRACT

Artificial Intelligence (AI) is rapidly advancing with a data-centered approach suitable for various domains. Nevertheless, AI faces significant challenges, particularly in data quality. Data collection from diverse sources can introduce quality issues that may threaten the development of AI-enabled systems. A growing concern in this context is the emergence of *data smells* – issues specific to the data used in building AI models, which can have long-term consequences. In this paper, we aim at enlarging the current body of knowledge on data smells, by proposing a two-step investigation into the matter. First, we updated an existing literature review in an effort of cataloguing the currently existing data smells and the tools to detect them. Afterward, we assess the prevalence of data smells and their correlation with data quality metrics. We identify a novel set composed of 12 data smells distributed across three additional categories. Secondly, we observe that the correlation between data smells and data quality is notably impactful, exhibiting a pronounced and substantial effect, especially in highly diffused data smell instances. This research sheds light on the complex relationship between data smells and data quality, providing valuable insights into the challenges of maintaining AI-enabled systems.

CCS CONCEPTS

• Software and its engineering → Software maintenance tools.

KEYWORDS

AI Technical Debt; Data Smells; Data Quality; Software Engineering for Artificial Intelligence, Empirical Software Engineering.

ACM Reference Format:

Gilberto Recupito, Raimondo Rapacciuolo, Dario Di Nucci, and Fabio Palomba. 2024. Unmasking Data Secrets: An Empirical Investigation into Data Smells and Their Impact on Data Quality. In *Conference on AI Engineering Software Engineering for AI (CAIN 2024)*, April 14–15, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3644815.3644960>



This work licensed under Creative Commons Attribution International 4.0 License.

CAIN 2024, April 14–15, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0591-5/24/04.
<https://doi.org/10.1145/3644815.3644960>

1 INTRODUCTION

Artificial Intelligence (AI) is more and more diffused nowadays, being used by individuals and companies to make informed decisions [54] and automate tasks that would typically be done by humans [37]. Indeed, AI-intensive systems, i.e., systems that embed artificial intelligence models and algorithms, have been recently deployed in multiple domains, with some recent applications showing highly efficient and accurate performance [30, 33].

However, the development of artificial intelligence-enabled systems differs from other types of software because the program and its effectiveness in solving a specific task heavily rely on the data and observations used to train models [2].

More specifically, AI-enabled systems are defined as a *system consisting of various software components, out of which at least one is an AI-specific component* [29]. In this context, data represents the primary source of producing business-oriented AI-enabled systems. Performing data analysis and validation is a crucial initial step for designers of machine learning components. Failure to properly analyze the training data may lead to model degradation [26]. Therefore, it is crucial to prioritize data quality to build a reliable and effective AI-enabled system. Data quality issues can arise for various reasons, e.g., data entry errors, inadequate data cleaning, or bias in the data. Addressing these issues requires a well-defined data quality management strategy that includes profiling, cleansing, and data enrichment [32]. While different tools and practices are available to support feature engineering and data transformation for managing AI pipelines [38], the need to improve the practices related to quality assurance is continuously increasing [9].

Data quality degradation could also lead to technical debt for the whole system [12]. Data debt, primarily when introduced by data quality issues or data anomalies, can strongly impact AI-enabled systems, degrading model performance and causing problems to all the subsequent phases involved in the pipeline [1].

In analyzing technical debt specific to data, *data smells* are represented using the analogy of code smells. As code smells are defined as *symptoms of poor design and implementation choices* [14], data smells are *data value-based indications of latent data quality issues caused by poor practices that may lead to problems in the future* [13]. While other types of data quality issues are investigated in research to be addressed [16, 18, 27, 49], the knowledge about the effect and the presence of data smells is still limited to the catalog provided by Foidl et al. [13].

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

4

Data changes. Data and assumptions we make about data can evolve over time in many ways for many different reasons. In general, mechanisms of how we process data or evaluate data quality today may need to be adapted tomorrow.

First, software, models, and hardware can change over time. Second, operator and user behavior can change over time (feature drift, concept drift). Finally, the environment and system requirements can change, influencing the data-quality requirements for the system.

Unfortunately, data-quality problems *can take a long time to notice*. They are usually very difficult to spot in traditional offline evaluations of prediction accuracy. For example, a project that trains/evaluates a classifier with outdated data may get promising accuracy results in offline evaluations, but may only notice that the model is inadequate for production once it is deployed.

In general, as studies have confirmed again and again, the later in a project a problem is discovered, the more expensive it is to fix, because we may have already committed to many faulty assumptions in the system design and implementation.

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

As such, proactive monitoring and alerting in production is a common and important strategy to detect drift. There are many different measures that can be observed. Here, we provide some ideas:

- 1 *Difference between inference data and past or training data distribution*, for instance through methods like Kullback-Leibler Divergence - which we mentioned already when introducing operational challenges.
- 2 *Number of outliers or low-confidence predictions*, for instance by using the so-called anomaly detection methods - which are unsupervised methods able to discover anomalous patterns in data.
- 3 *Difference in influential features and feature importance*; various *explainability techniques* can identify what features are important for a prediction or which features a model generally relies on the most. We can use distribution difference measures to see whether the model starts relying on different features.
- 4 *Number of wrong predictions*: If the correctness of a prediction can be determined or approximated from telemetry data, observing any accuracy measure over time can provide insights into the impact of all forms of drift.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

When it turns to the model, additional issues may arise. These are mainly due to the concept of “*technical debt*”, a metaphor used to express the hidden costs in the mid/long term due to the application of quick workarounds rather than longer but better approaches.

Just like taking out a loan provides the borrower with money that can be used immediately but needs to be repaid later with accumulated interest, in the software development analogy, certain actions *provide short-term benefits*, such as more features and earlier releases, but come *at the cost of lower productivity, needed rework, or additional operating costs* later.

Technical debt. A design or implementation constructs that are expedient in the short term, but set up a technical context that can make a future change more costly or even impossible.

The power of the technical debt metaphor comes from its simplicity and ease of communicating implications with non-developers, including managers and customers - it is easier to explain why developers want to spend time on infrastructure and maintenance work now rather than spending all their time on completing features...

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:

	Reckless	Prudent
Deliberate	<i>“We don’t have time to plan for mistakes.”</i>	<i>“We must release now before the competition and deal with slow manual releases for now.”</i>
Inadvertent	<i>“What’s data drift?”</i>	<i>“We didn’t know about data versioning and in retrospect it’s good we didn’t waste a lot of effort on it early on.”</i>

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:

Reckless

“We don’t have time to plan for mistakes.”

Prudent

“We must release now before the competition and deal with slow manual releases for now.”

Deliberate

Most financial debt comes from a deliberate decision to take out a loan. Also, technical debt can come from **deliberate decisions to take a shortcut**, knowing that it will cost them later.

“We didn’t know about data versioning and in retrospect it’s good we didn’t waste a lot of effort on it early on.”

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:

In contrast, developers may **inadvertently** make decisions that will incur later costs without thinking through the benefit-cost trade-off. Some developers only learn about how they could have built the system better at an earlier stage once they face a problem and explore possible solutions.

Student

“We must release now before the competition and deal with slow manual releases for now.”

Inadvertent

“What’s data drift?”

“We didn’t know about data versioning and in retrospect it’s good we didn’t waste a lot of effort on it early on.”

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:

In the financial setting, many loans are used for investments that provide a chance for high or long-term returns. In the same way, engineering decisions that favor short-term benefits over long-term maintenance costs may be a prudent, good investment. For example, we may want to speed up the release to be the first on the market and acquire customers.

What's data unit?

Prudent

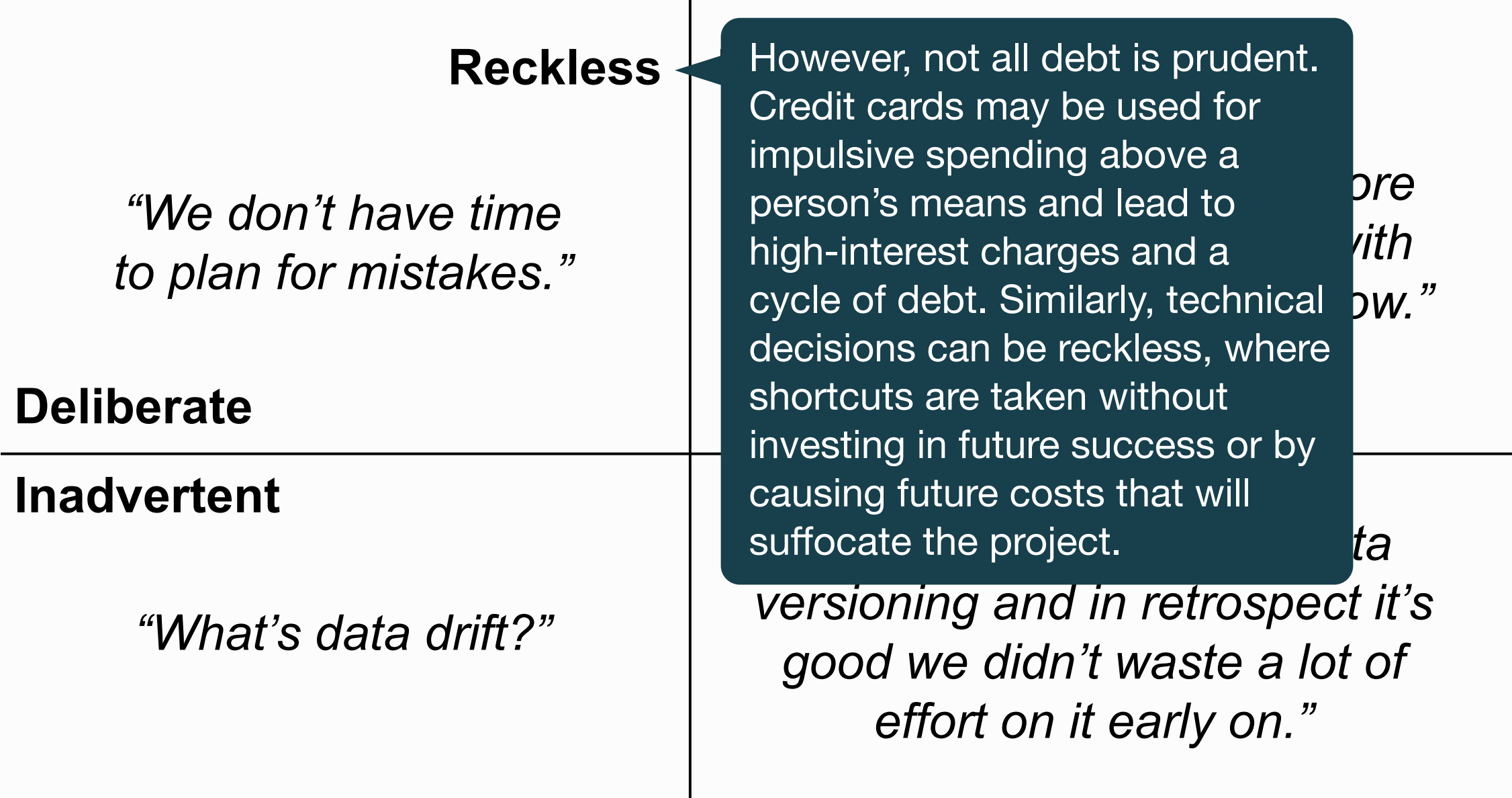
“We must release now before the competition and deal with slow manual releases for now.”

“We didn’t know about data versioning and in retrospect it’s good we didn’t waste a lot of effort on it early on.”

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:



Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Martin Fowler, one of the fathers of technical debt research, provided a useful, more nuanced view of different forms of technical debt considering two dimensions:

	Reckless	Prudent
Deliberate	<i>"We don't have time to plan for mistakes."</i>	<i>"We must release now before the competition and deal with slow manual releases for now."</i>
Inadvertent	<i>"What's data drift?"</i>	<i>"We didn't know about data versioning and in retrospect it's good we didn't waste a lot of effort on it early on."</i>

Ideally, all technical debt is deliberate and prudent. Ideally, developers consider their options and deliberately decide that certain benefits, such as a faster release, are worth the anticipated future costs.

In practice though, many forms of technical debt will be inadvertent and reckless. Not knowing better or being under immense stress to deliver quickly, developers may take massive shortcuts that become very expensive later.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

AI makes it very easy to inadvertently accumulate massive amounts of technical debt, especially when inexperienced teams try to deploy products with artificial intelligence components in production.

Hidden Technical Debt in Machine Learning Systems

In this seminal paper, Sculley and other Google engineers describe many ways machine learning projects incur additional forms of maintenance costs beyond that of traditional software projects and frame them as technical debt.

Abstract

Machine learning offers a fantastically powerful toolkit for building useful complex prediction systems quickly. This paper argues it is dangerous to think of these quick wins as coming for free. Using the software engineering framework of *technical debt*, we find it is common to incur massive ongoing maintenance costs in real-world ML systems. We explore several ML-specific risk factors to account for in system design. These include boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies, configuration issues, changes in the external world, and a variety of system-level anti-patterns.

Quick link to the paper:

<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

AI makes it very easy to inadvertently accumulate massive amounts of technical debt, especially when inexperienced teams try to deploy products with artificial intelligence components in production.

First of all, using AI/ML where it is not necessary creates technical debt: using AI/ML can seem like a quick and easy solution to a problem, especially when AI is hyped in the media and by consultants, but it can come with long term costs that the initial developers may not anticipate.

As we discuss throughout this course, engineering production-quality AI components can be expensive and induce long-term maintenance costs.

In addition, poor engineering practices in AI-enabled projects can introduce substantial technical debt.

Throughout this course, we are discussing many good engineering practices that reduce maintenance costs and risks. These practices address emerging problems and are designed to avoid long-term costs, but **they all require an up-front investment** to perform some analysis or adopt or develop nontrivial infrastructure.

Learning Systems

Google engineers
projects incur additional
traditional software
technical debt.

Building useful com-
puter systems is dan-
gerous to think of
engineering framework
as a one-time cost of
doing maintenance
and ignoring specific risk factors to
avoid, entanglement,
complexity, configuration
level anti-patterns.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

Traditional software engineering showed that strong abstraction using encapsulation and modular design help create maintainable code that is easy to modify and enhance.

Problem: Machine learning is required when the desired behavior *cannot be effectively expressed in software logic* without dependency on external data. Hence, it is hard to define strict abstractions.

Let's consider the problems caused by entanglement.

Consider a system that uses features $x_1, \dots, x_n$ in a model. What happens if you need to change one of those features?

The CACE principle: *Changing Anything Changes Everything*.

The importance, weights, or use of the remaining $n-1$ features may all change. Note that CACE applies not only to inputs, but also to hyper-parameters, learning settings, sampling methods, convergence thresholds, data selection, and essentially every other possible tweak.

While this is something unavoidable, the potential impact of entanglement is huge: for instance, think at how much the accuracy of the model might change!

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

Traditional software engineering showed that strong abstraction using encapsulation and modular design help create maintainable code that is easy to modify and enhance.

Problem: Machine learning is required when the desired behavior *cannot be effectively expressed in software logic* without dependency on external data. Hence, it is hard to define strict abstractions.

Solution #1: Isolate models and serve ensembles. Whenever you can - and this may not happen so often - you might want to decompose your model in multiple models that are combined together by means of ensemble methods, e.g., creating smaller models composed of subset of features that are later combined. In this way, changes to one model may have a limited impact on the others and, more importantly, on the prediction capabilities of the model.

Solution #2: Detecting changes in prediction behavior as they occur. High-dimensional visualization tools might be useful here to allow data scientists to quickly see effects across many dimensions and slicings.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

Traditional software engineering showed that excessive coupling between objects makes the source code more complex and harder to maintain and understand.

Problem: Similar effects have been noticed when considering the data dependencies of machine learning systems.

The presence of correlated variables is one of the technical debt types connected to data dependency. Yet, there are snakiest cases that cannot be handled during preprocessing.

Let's consider the problems caused by underutilized data dependencies. These are data that provide little incremental modeling benefit that have the effect of making the machine learning system unnecessarily vulnerable to change.

- ϵ -features: It is always tempting to improve model accuracy even when the accuracy gain is very small or when the complexity overhead might be high.
- Legacy features: These are features included in the model at some point but that become redundant during maintenance and evolution, e.g., when new features are included in the model.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

Traditional software engineering showed that excessive coupling between objects makes the source code more complex and harder to maintain and understand.

Problem: Similar effects have been noticed when considering the data dependencies of machine learning systems.

Let's consider the problems caused by underutilized data dependencies. These are data that provide little incremental modeling benefit that have the effect of making the machine learning system unnecessarily vulnerable to change.

- ϵ -features: It is always tempting to improve model accuracy even when the accuracy gain is very small or when the complexity overhead might be high.
- Legacy features: These are features included in the model at some point but that become redundant during maintenance and evolution, e.g., when new features are included in the model.

Solution: Perform exhaustive leave-one-feature-out evaluations. These should be run regularly to identify and remove unnecessary features.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

One of the key features of live machine learning systems is that they often end up influencing their own behavior if they update over time, making hard to isolate their individual properties.

Problem: This is what we call *feedback loop*, a form of analysis debt where you cannot easily assess the behavior of a model before releasing it.

Let's consider an example: You have two smart systems operating on a web application: the first selects products to show, the second selects the related reviews.

Improving one system may lead to changes in behavior in the other, as users begin clicking more or less on the other components in reaction to the changes.

This is a case of *hidden* feedback loop: a solution is still unknown, even if this problem may possibly lead to catastrophic consequences. For instance, consider two stock-market prediction models from two different investment companies. Improvements (or, more scarily, bugs) in one may influence the bidding and buying behavior of the other.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

As a matter of fact, only a tiny fraction of the code in many machine learning systems is devoted to learning or prediction. Yet, even this code may smell bad.

Problem: As any source code, even machine learning code is written by humans who apply design decisions based on contrasting goals and pressing deadlines.

Note that this is the category of technical debt with more types. On the one hand, it is easier to make code complex. On the other hand, the other technical debt types are more conceptual and therefore harder to classify and solve.

Let's consider the example of the pipeline jungle.

This often appears in the data preparation step of the machine learning pipeline. Indeed, the amount of data considered within the system might evolve organically, e.g., because of new information sources identified.

Without care, the resulting system for preparing data in an ML-friendly format may become a jungle of scrapes, joins, and sampling steps, often with intermediate files output. This makes maintenance harder, but also more fault-prone.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

As a matter of fact, only a tiny fraction of the code in many machine learning systems is devoted to learning or prediction. Yet, even this code may smell bad.

Problem: As any source code, even machine learning code is written by humans who apply design decisions based on contrasting goals and pressing deadlines.

Let's consider the example of the pipeline jungle.

This often appears in the data preparation step of the machine learning pipeline. Indeed, the amount of data considered within the system might evolve organically, e.g., because of new information sources identified.

Without care, the resulting system for preparing data in an ML-friendly format may become a jungle of scrapes, joins, and sampling steps, often with intermediate files output. This makes maintenance harder, but also more fault-prone.

Solution: Think holistically about data collection and feature extraction or even refactor the development community! This anti-pattern might be the result of the integration of multiple teams.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Detecting technical debt is still an active research field. Up to now, only some preliminary attempts have been proposed.

Let's consider again the case of Pipeline Jungle: you may want to identify chaotic pipelines—those characterized by excessive complexity, redundant steps, or unclear data flows. Which options do we have?

Code review can be a valuable instrument for identifying issues such as technical debt or suboptimal design decisions. However, several challenges arise: code review is applied to **individual changes in isolation**, hence **it may miss the broader architectural or systemic issues**—what we might call the “big picture”. This forces reviewers to mentally reconstruct the overall context, which can be both time-consuming and error-prone.

Moreover, effective code review in the context of complex pipelines requires reviewers to possess not only a **deep understanding of the entire pipeline structure** but also **specific expertise in identifying signs of technical debt and architectural erosion**.

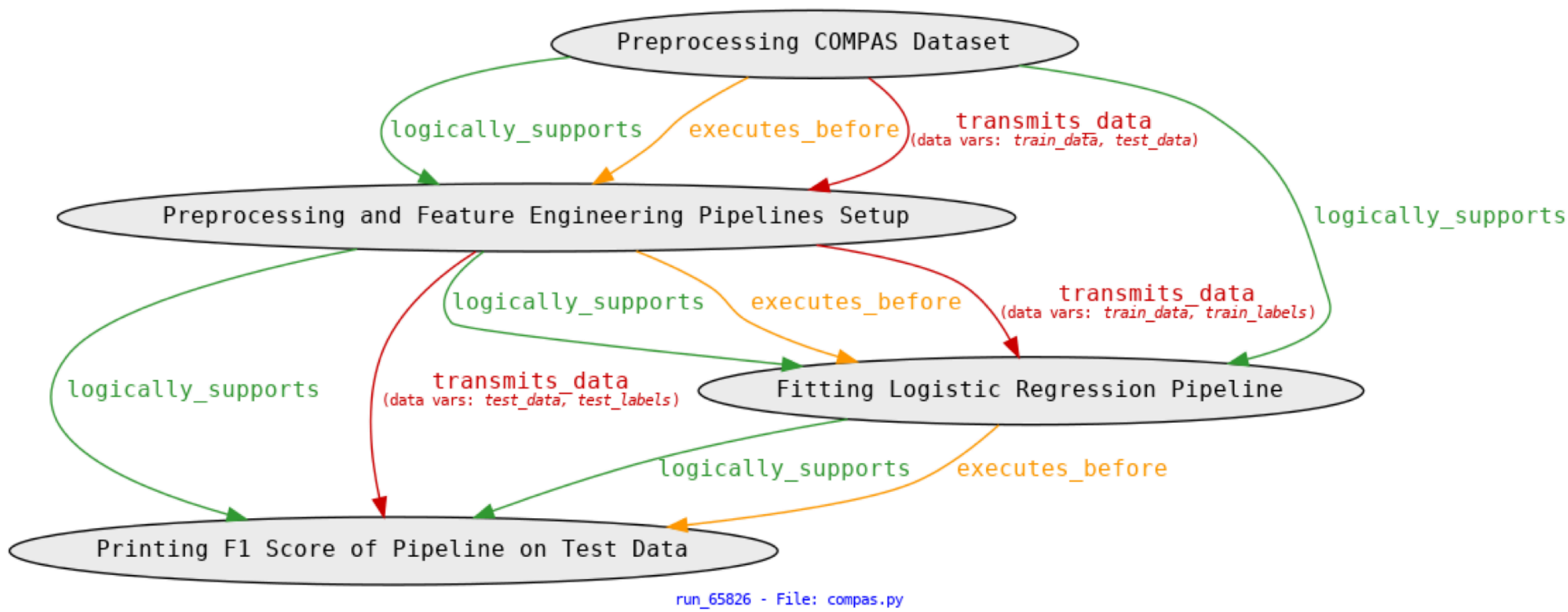
These challenges make the identification of this technical debt particularly hard; similar issues might be discussed for other technical debt items. Hence, some form of automated tool may be worthwhile. However, how can we automate the identification of some complex analysis that require full-pipeline considerations?

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

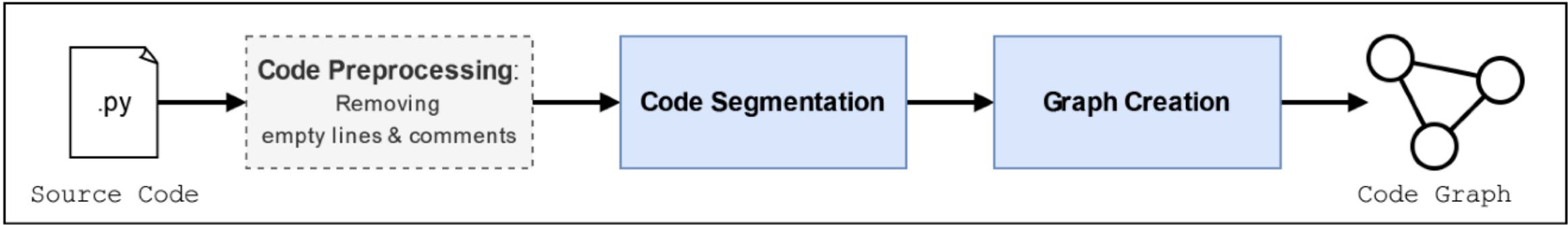
GILBERTO RECUPITO, University of Salerno, Italy
DOMINIK OPITZ, German Aerospace Center (DLR), Germany
TOBIAS HECKING, German Aerospace Center (DLR), Germany
DARIO DI NUCCI, University of Salerno, Italy
FABIO PALOMBA, University of Salerno, Italy
MICHAEL FELDERER, German Aerospace Center (DLR) University of Cologne, Germany

Machine Learning (ML) is increasingly used across diverse application domains, offering powerful capabilities for intelligent services. As ML integrates into larger software systems, it adds complexity, making maintenance and evolution challenging. ML pipelines manage the lifecycle of these models but are often difficult to understand, maintain, and adapt due to limited support from existing orchestration tools. This paper presents a novel approach, coined CLAIRE (Collaborative LLMs Approach for Intelligent Representation Extraction

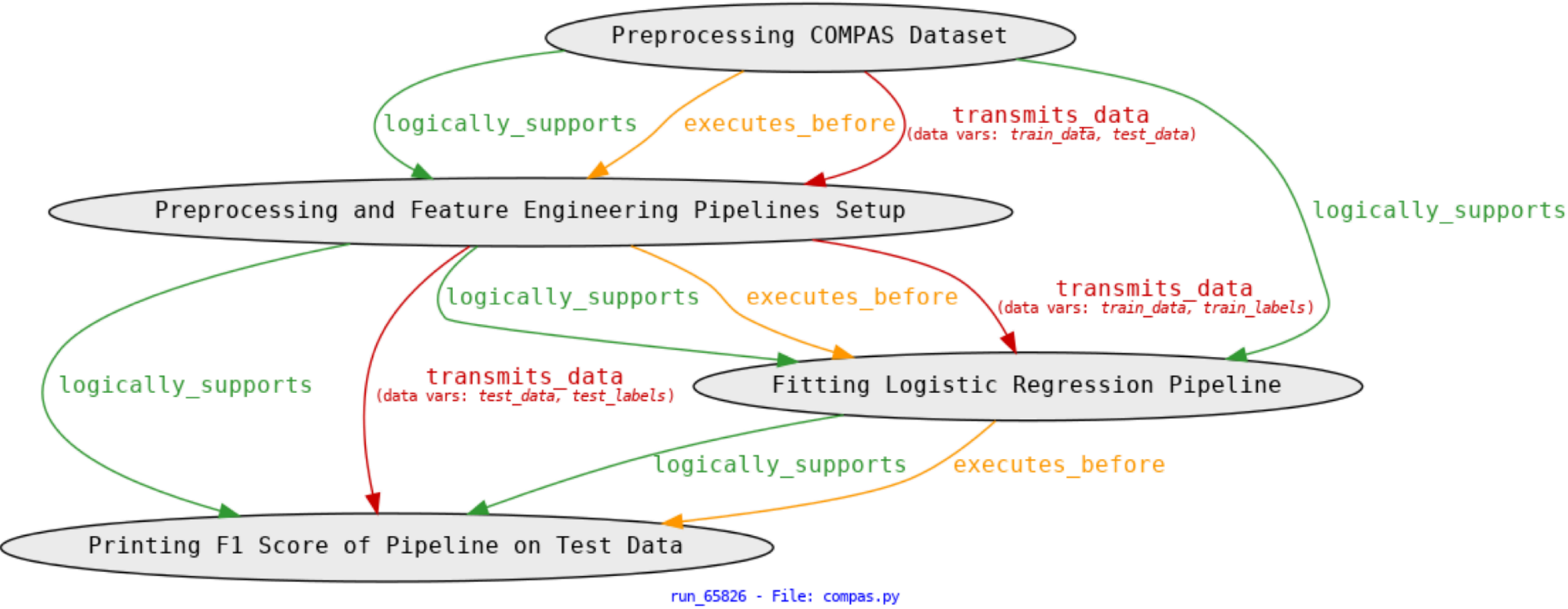


Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For



understand, maintain, and adapt due to limited support from existing orchestration tools. This paper presents a novel approach, coined CLAIRE (Collaborative LLMs Approach for Intelligent Representation Extraction



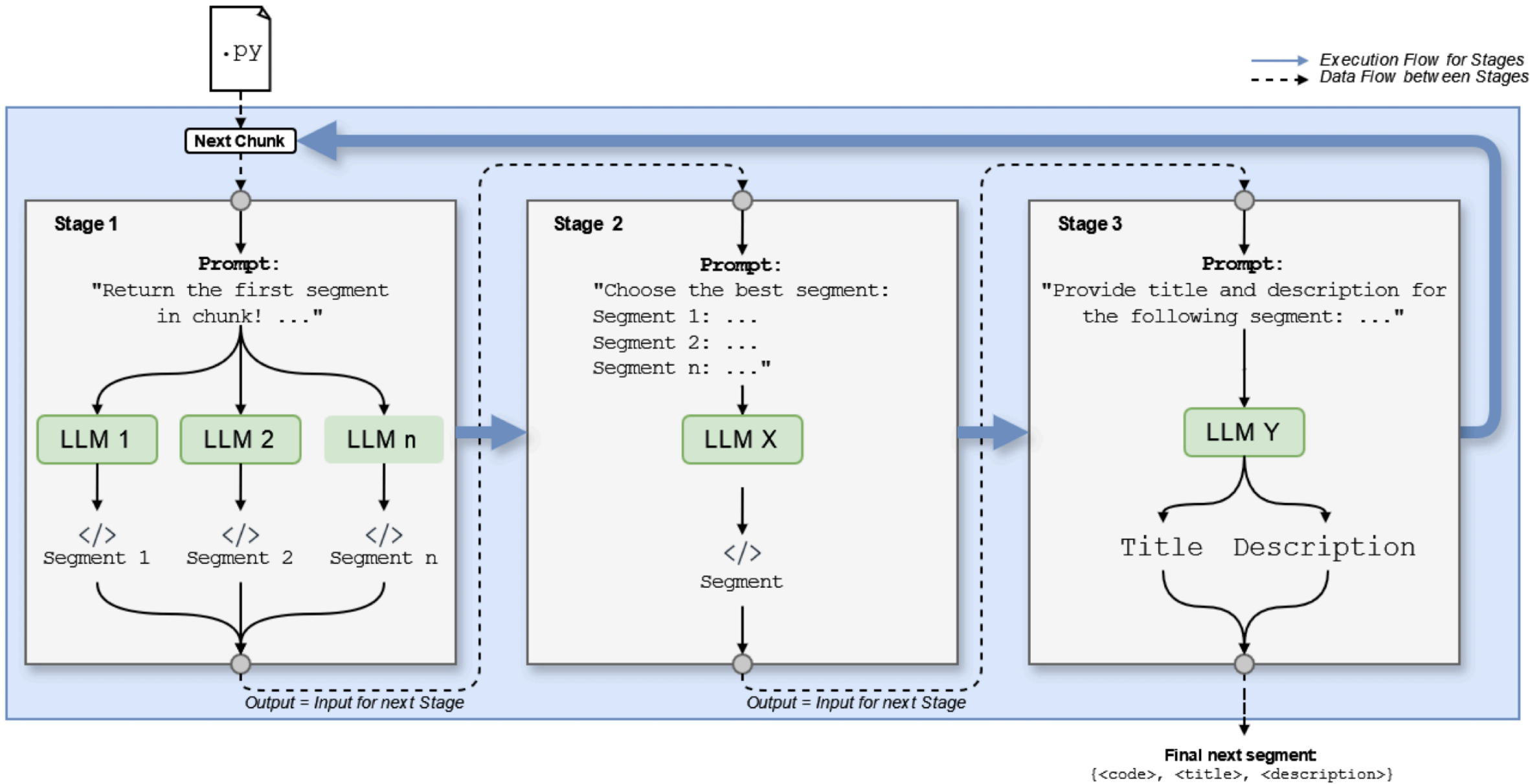
Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

GILBERTO RECUPITO, University of Salerno, Italy

DOMINIK OPITZ, German Aerospace Center (DLR), Germany

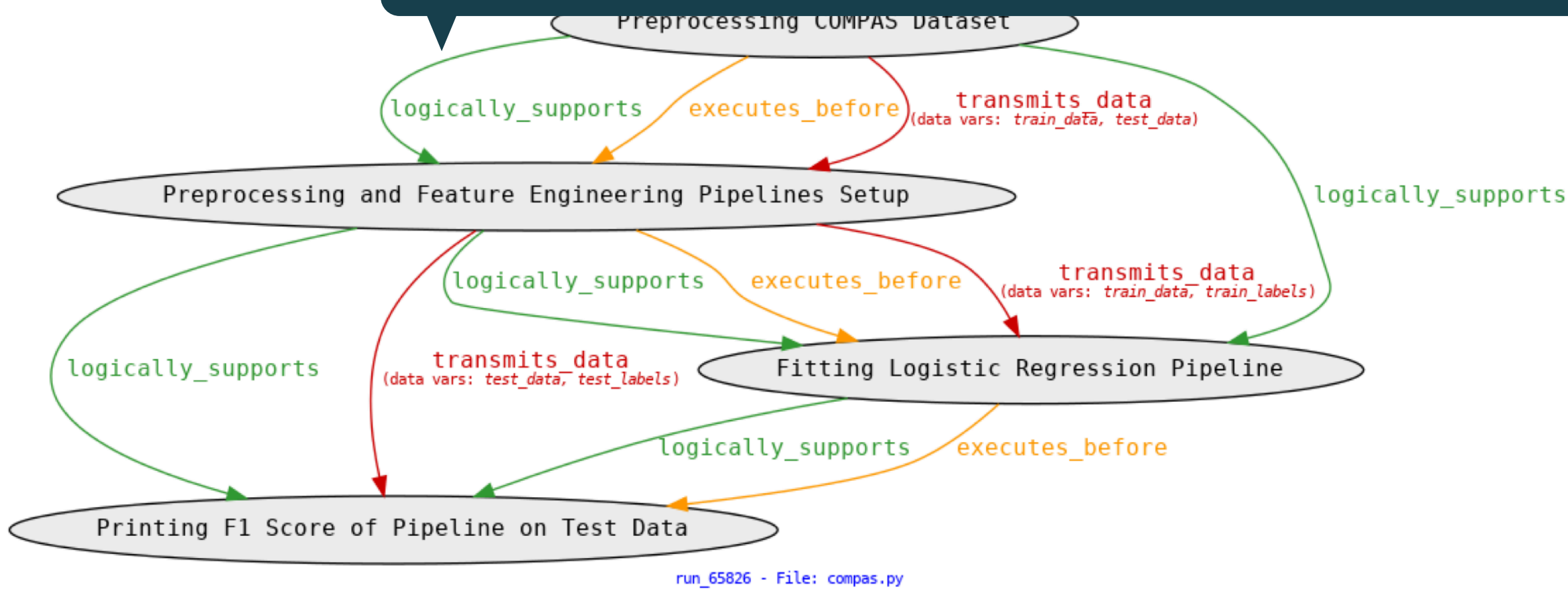


Quality Assurance of Artificial Intelligence Solutions: What to Look For

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

GILBERTO RECUPITO, University of Salerno, Italy
DOMINIK OPITZ, German Aerospace Center (DLR), Germany
TOBIAS HECKING, German Aerospace Center (DLR), Germany
DARIO DI NUCCI, University of Salerno, Italy
FABIO PALOMBA, University of Salerno, Italy

Logical edge: Two segments are connected by a logical edge if the segments use each others variables. Therefore, a single node can be the source and/or target node of multiple logical dependencies.



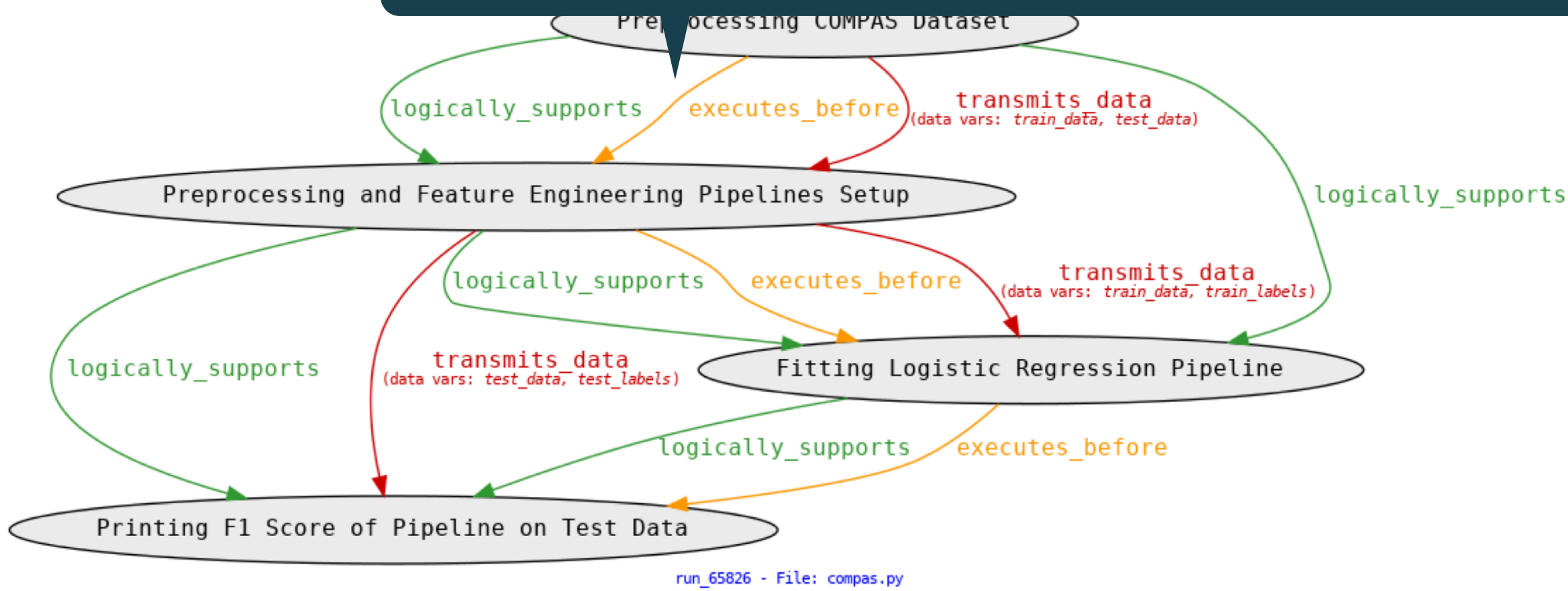
Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

GILBERTO RECUPITO, University of Salerno, Italy
DOMINIK OPITZ, German Aerospace Center (DLR), Germany
TOBIAS HECKING, German Aerospace Center (DLR), Germany
DARIO DI NUCCI, University of Salerno, Italy
FABIO PALOMBA, University of Salerno, Italy

Execution edge: As the name suggests, this edge represents the order of execution of segment nodes. It provides a definite execution order for each of the nodes.

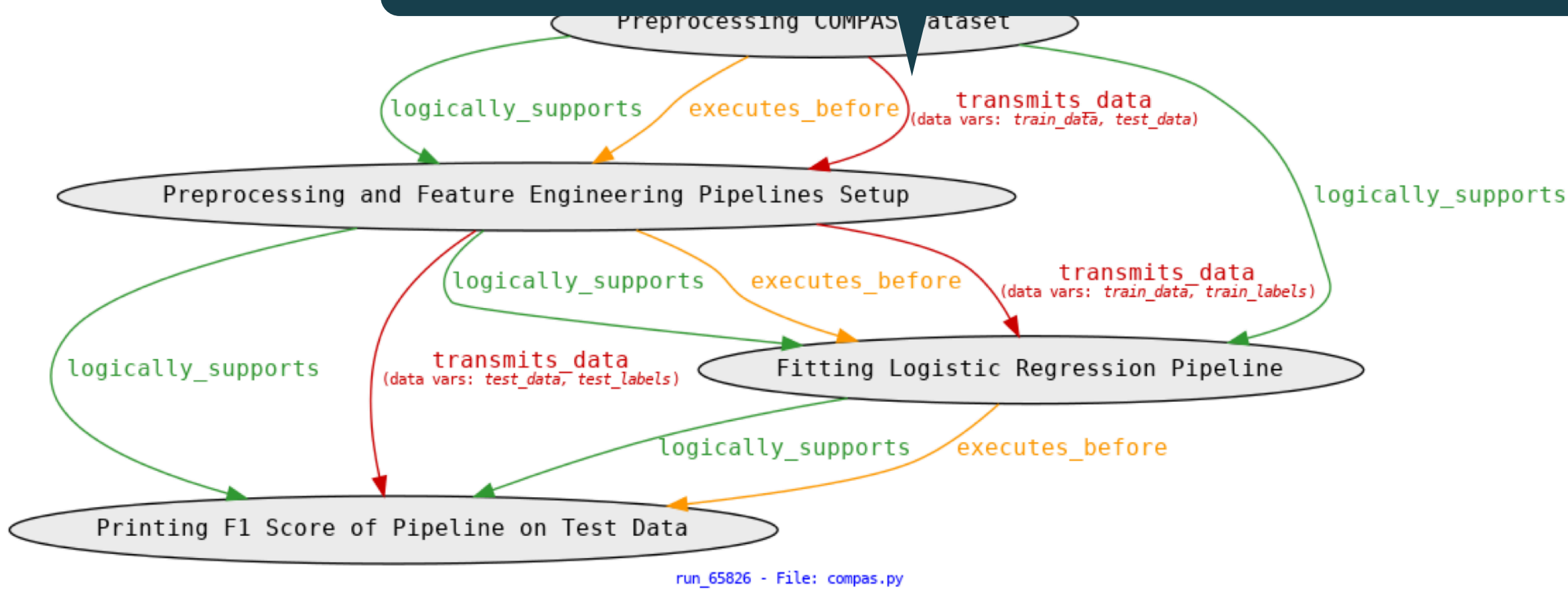


Quality Assurance of Artificial Intelligence Solutions: What to Look For

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

GILBERTO RECUPITO, University of Salerno, Italy
DOMINIK OPITZ, German Aerospace Center (DLR), Germany
TOBIAS HECKING, German Aerospace Center (DLR), Germany
DARIO DI NUCCI, University of Salerno, Italy
FABIO PALOMBA, University of Salerno, Italy

Dataflow edge: It represents the flow of data between two operations or components in a system, typically connecting the output of one node (e.g., a task or function) to the input of another.



Software Engineering for Artificial Intelligence: Quality Assurance

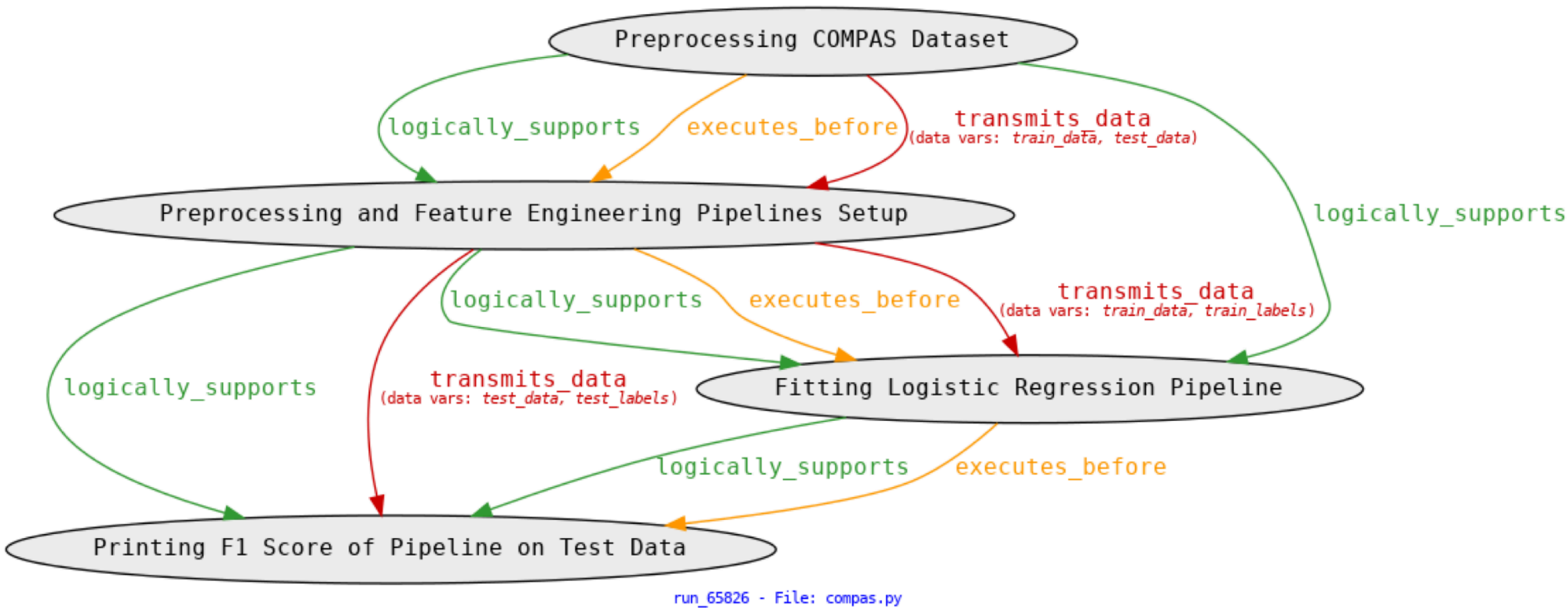
Quality Assurance of Artificial Intelligence Solutions: What to Look For

A similar representation may be used to detect Pipeline Jungle and other technical debt that requires the static analysis of whole pipelines.

Automated Logical Graph Representation of Machine Learning Pipelines Using Collaborative Language Models

GILBERTO RECUPITO, University of Salerno, Italy
DOMINIK OPITZ, German Aerospace Center (DLR), Germany
TOBIAS HECKING, German Aerospace Center (DLR), Germany
DARIO DI NUCCI, University of Salerno, Italy
FABIO PALOMBA, University of Salerno, Italy
MICHAEL FELDERER, German Aerospace Center (DLR) University of Cologne, Germany

Machine Learning (ML) is increasingly used across diverse application domains, offering powerful capabilities for intelligent services. As ML integrates into larger software systems, it adds complexity, making maintenance and evolution challenging. ML pipelines manage the lifecycle of these models but are often difficult to understand, maintain, and adapt due to limited support from existing orchestration tools. This paper presents a novel approach, coined CLAIRE (Collaborative LLMs Approach for Intelligent Representation Extraction



Software Engineering for Artificial Intelligence: Verification and Validation

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Basics

Quality assurance measures the level of quality in a service/product by means of manual/automated mechanisms at every stage of the process of delivery or production.

QA approaches for software can be roughly divided into three large groups:

- 1 *Dynamic approaches* execute the software, including traditional testing and dynamic program analyses.
- 2 *Static approaches* analyze code, documentation, designs, or other software artifacts without executing them, including code review, code linting, static data-flow analysis, and formal verification.
- 3 *Process evaluation approaches* analyze how the software was produced rather than the software artifacts, for example, assuring that requirements are documented and known bugs are tracked.

Many QA activities can be automated. For example, the execution large test suites automatically after every change to a code base. At the same time, some QA approaches rely on manual steps or are entirely manual, such as asking experts to review design documents for performance bottlenecks or security weaknesses.

QA often focuses on *functional correctness* to evaluate whether the software produces the expected outputs according to some specification. Beyond functional correctness, there are many other *qualities of interest* for components or the system as a whole.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

As such, proactive monitoring and alerting in production is a common and important strategy to detect drift. There are many different measures that can be observed. Here, we provide some ideas:

- 1 *Difference between inference data and past or training data distribution*, for instance through methods like Kullback-Leibler Divergence - which we mentioned already when introducing operational challenges.
- 2 *Number of outliers or low-confidence predictions*, for instance by using the so-called anomaly detection methods - which are unsupervised methods able to discover anomalous patterns in data.
- 3 *Difference in influential features and feature importance*; various *explainability techniques* can identify what features are important for a prediction or which features a model generally relies on the most. We can use distribution difference measures to see whether the model starts relying on different features.
- 4 *Number of wrong predictions*: If the correctness of a prediction can be determined or approximated from telemetry data, observing any accuracy measure over time can provide insights into the impact of all forms of drift.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

Going deeper on AI technical debt, it captures problems that may appear over the whole AI pipeline, from data engineering to model deployment and retraining.

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

Traditional software engineering showed that strong abstraction using encapsulation and modular design help create maintainable code that is easy to modify and enhance.

Problem: Machine learning is required when the desired behavior *cannot be effectively expressed in software logic* without dependency on external data. Hence, it is hard to define strict abstractions.

Let's consider the problems caused by entanglement.

Consider a system that uses features $x_1, \dots, x_n$ in a model. What happens if you need to change one of those features?

The CACE principle: *Changing Anything Changes Everything*.

The importance, weights, or use of the remaining $n-1$ features may all change. Note that CACE applies not only to inputs, but also to hyper-parameters, learning settings, sampling methods, convergence thresholds, data selection, and essentially every other possible tweak.

While this is something unavoidable, the potential impact of entanglement is huge: for instance, think at how much the accuracy of the model might change!

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

Data quality is a challenge in many projects; there are a number of considerations to make in this respect.

- 3 *Data quality criteria*. Because of the importance of data quality, multiple criteria exist to evaluate the intrinsic properties of data - some should be already known.

Criterion	Definition	Rationale
Accuracy	The data was recorded correctly.	Inaccurate data can lead to incorrect analyses, decisions, or predictions.
Completeness	All relevant data was recorded.	Incomplete data can result in biased or misleading outcomes.
Uniqueness	All entries are recorded once.	Duplicate records can skew results and waste resources.
Consistency	The data agrees with each other.	Inconsistent data across systems or records can cause confusion and errors.
Currentness	The data is kept up to date.	Outdated data can lead to irrelevant or incorrect decisions.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

In an AI-enabled system, we are basically interested in validating the two key elements of AI components, namely **data** and **model**.

When it turns to the model, additional issues may arise. These are mainly due to the concept of "*technical debt*", a metaphor used to express the hidden costs in the mid/long term due to the application of quick workarounds rather than longer but better approaches.

Just like taking out a loan provides the borrower with money that can be used immediately but needs to be repaid later with accumulated interest, in the software development analogy, certain actions *provide short-term benefits*, such as more features and earlier releases, but come at the *cost of lower productivity, needed rework, or additional operating costs* later.

Technical debt. A design or implementation constructs that are expedient in the short term, but set up a technical context that can make a future change more costly or even impossible.

The power of the technical debt metaphor comes from its simplicity and ease of communicating implications with non-developers, including managers and customers - it is easier to explain why developers want to spend time on infrastructure and maintenance work now rather than spending all their time on completing features...

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: What to Look For

AI makes it very easy to inadvertently accumulate massive amounts of technical debt, especially when inexperienced teams try to deploy products with artificial intelligence components in production.

First of all, using AI/ML where it is not necessary creates technical debt: using AI/ML can seem like a quick and easy solution to a problem, especially when AI is hyped in the media and by consultants, but it can come with long term costs that the initial developers may not anticipate.

As we discuss throughout this course, engineering production-quality AI components can be expensive and induce long-term maintenance costs.

In addition, poor engineering practices in AI-enabled projects can introduce substantial technical debt. Throughout this course, we are discussing many good engineering practices that reduce maintenance costs and risks. These practices address emerging problems and are designed to avoid long-term costs, but **they all require an up-front investment** to perform some analysis or adopt or develop nontrivial infrastructure.



Software Engineering for Artificial Intelligence

Quality Assurance of Artificial Intelligence Solutions

